

Styx AI — Plan

Overview

AI-assisted tooling for the Styx/NiWrap ecosystem, covering two directions: an **MCP server** for tool discovery and code generation, and an **agent pipeline** for automated descriptor creation and validation.

The descriptor language is moving from Boutiques JSON to **argtype**. Boutiques becomes a compilation target. The pipeline becomes:

```
source code → agents → argtype → Styx compiler → {Python, TS, R, Boutiques, CWL, MCP, ...}
```

argtype is ~10x more concise than Boutiques JSON, formally regular (decidable validation/completion), and fits entirely in an LLM system prompt (~2–3k tokens). This makes it far more suitable for both LLM generation and human review.

MCP Server

NiWrap has ~1900 descriptors — too many to expose as individual MCP tools. Instead, use a **search-then-invoke** pattern with 2–3 meta-tools:

- `search_tools(query)` — search by keyword/package, returns names + descriptions.
- `get_tool_schema(tool_id)` — full typed signature, parameter docs, outputs.
- `generate_invocation(tool_id, params)` — optional; generates niwrap Python code.

The server generates code, not executes tools. Execution happens in the user's environment. Works in Claude Code, Cursor, or any MCP client.

Effort: 2–3 days.

Agent Pipeline

Four agents in a loop: **Explorer** → **Author** → **Validator** → **Refinement**.

Explorer

Navigates package source code to extract CLI structure. Outputs structured JSON (not argtype). Has filesystem/grep tools over cloned repos.

Per-package strategies needed. ANTs is a good example of the hard case — C++ with complex template-based argument handling across dozens of binaries, inconsistent help text, and implicit parameter dependencies. Contrast with packages where argument parsing can be extracted statically.

Effort: ~1 week.

Author

Translates explorer output into argtype. Gets the full spec in its system prompt + few-shot examples from the same package. Sonnet-class model is sufficient.

Effort: 2–3 days.

Validator

The quality gate. Multiple layers:

Layer	LLM needed?
Grammar validation (parse the argtype)	No
Help text comparison (tool -help vs descriptor)	Yes
Command-line round-tripping (random valid params → verify)	No
Smoke test (run with OpenNeuro data, check exit code + outputs)	No
Output template verification (predicted vs actual paths)	No
Failure diagnosis (interpret stderr, suggest fixes)	Yes (Opus)

Real data from **OpenNeuro** (e.g. ds000114 — T1w + DWI + fMRI) enables end-to-end behavioral validation. Most time goes to edge cases: tools needing specific env vars, non-deterministic output names, tools that write to unexpected locations.

Effort: 1–2 weeks.

Refinement Loop

Structured error reports from validator feed back to author. Caps at N retries, then flags for human review.

Effort: 2–3 days.

Implementation

Thin orchestration on the **Anthropic API** directly — no heavy frameworks. Each agent is a function with a focused prompt and tool set. Batch API for large runs.

```
async def process_tool(tool_binary, package, container_image):
    source_analysis = await explore(tool_binary, package)
    descriptor = await author(source_analysis, package)

    for attempt in range(max_refinements):
        report = await validate(descriptor, container_image, test_data)
        if report.passed:
            break
        descriptor = await refine(descriptor, report)

    return descriptor
```

Estimated API cost for a full run (~1900 tools): \$200–500.

Roadmap

Phase	Work	Effort
0	argtype parser / Styx frontend	2–4 weeks (critical path)
1	Validator against existing descriptors	1–2 weeks
2	MCP server	2–3 days

Phase	Work	Effort
3	Explorer + Author agents	~1.5 weeks
4	Refinement loop	2–3 days

Start with Phase 1. The validator delivers immediate value (bugs in existing 1900 descriptors) and builds the test infrastructure the generation agents depend on. MCP server is independent and can happen in parallel.

Total: ~6–8 weeks.